

EECS471 Final Project
Abdullah Hendy (ahendy)
Shahmir Mahmud (shahmirm)

Optimization Summary:

- Explicit “im2col” conversion of activation tensor, $\mathbf{x}$, and weight, $\mathbf{w}$.
- Moved to implicit implementation of “im2col” fused with tiled GEMM kernel.
- Relaxed the tiled GEMM kernel to have rectangular tile sizes and used C++ templates.
- Created a separate tiled convolution kernel for Layer 1.
- Moved to a tensor core based kernel for Layer 2.
- Further optimize the tensor core based kernel for Layer 2.

Optimization 1: Explicit “im2col” GEMM Layers 1 and 2

- Unroll the whole X (similar to class example but done for all images instead of just one) from (B, C, H, W) into (C * K * K, B * H_out * W_out) in a **separate** kernel. The output unrolled X, $\mathbf{x}_{unrolled}$, was cached into a separate Tensor.
 - The following unrolling **pseudo code logic** is the one used in this kernel and in all subsequent kernels.

```
function X_UNROLL_KERNEL(x, x_unrolled, B, C, H, W, K, H_out, W_out):
    // Each thread corresponds to one element in the unrolled matrix
    idx = global_thread_id()
    // Total columns are all output positions across all batch elements
    num_cols = B * H_out * W_out
    row = idx / num_cols
    col = idx % num_cols
    // Decode the row index → (channel c, kernel offset p, q)
    c = row / (K*K)
    p = (row / K) % K
    q = row % K
    // Decode the column → (batch b, output h_out, w_out)
    b = col / (H_out * W_out)
    h_out = (col / W_out) % H_out
    w_out = col % W_out
    // Convert output location + kernel offset → actual input coordinates
    h_in = h_out + p
    w_in = w_out + q

    x_unrolled[idx] = x[b, c, h_in, w_in]
```

- Unroll W using `w.view({M, C * K * K});` to test the kernel quickly.
- Matrix multiplication using the tiled matrix multiplication from assignment 1
- Doesn't fit on the Volta GPU (16GB VRAM) since the separate huge variable ***x_unrolled*** is huge. Alone, it's 15.9 GB, for layer 2 considering its size of $(C * K * K, B * H_{out} * W_{out})$
- For this kernel to work, lowering the batch size was the solution, which is not allowed
- The following is output confirming the ***CUDA out of memory*** error for B=10000.

```

4 Loading fashion-mnist data...
5 Warming up CUDA kernels...
6 Traceback (most recent call last):
7   File "/gpfs/accounts/eecs471f25_class_root/eecs471f25_class/ahendy/final_project/lib/submission.py", line 141, in <module>
8     File "/gpfs/accounts/eecs471f25_class_root/eecs471f25_class/ahendy/final_project/lib/submission.py", line 125, in main
9     File "/sw/pkgsg/arc/python/3.10.4/pytorch/2.0.1/lib/python3.10/site-packages/torch/utils/_contextlib.py", line 115, in decorate_context
10    File "/gpfs/accounts/eecs471f25_class_root/eecs471f25_class/ahendy/final_project/lib/submission.py", line 87, in evaluate
11    File "/sw/pkgsg/arc/python/3.10.4/pytorch/2.0.1/lib/python3.10/site-packages/torch/nn/modules/module.py", line 1501, in _call_impl
12    File "/sw/pkgsg/arc/python/3.10.4/pytorch/2.0.1/lib/python3.10/site-packages/torch/nn/modules/container.py", line 217, in forward
13    File "/sw/pkgsg/arc/python/3.10.4/pytorch/2.0.1/lib/python3.10/site-packages/torch/nn/modules/module.py", line 1538, in _call_impl
14    File "/gpfs/accounts/eecs471f25_class_root/eecs471f25_class/ahendy/final_project/lib/submission.py", line 57, in forward
15    File "/sw/pkgsg/arc/python/3.10.4/pytorch/2.0.1/lib/python3.10/site-packages/torch/_ops.py", line 502, in _call_
16    torch.cuda.OutOfMemoryError: CUDA out of memory. Tried to allocate 15.97 GiB (GPU 0; 15.77 GiB total capacity; 1.08 GiB already allocated; 12.86 GiB free; 2.54
    GiB reserved in total by PyTorch) If reserved memory is >> allocated memory try setting max_split_size_mb to avoid fragmentation.  See documentation for Memory
    Management and PYTORCH_CUDA_ALLOC_CONF

```

Optimization 2: Implicit GEMM (Unroll on the fly) Layers 1 and 2

- Inspired by https://docs.nvidia.com/cutlass/latest/media/docs/cpp/implicit_gemm_convolution.html, the assignment 1 tiled matrix multiplication and the unrolling logic (from optimization 1) were fused to remove the extra step/tensor of ***x_unrolled***.
- This optimization version fit on the GPU
- Similar to optimization 0, the same kernel was applied to both layers.
- The following shows the latency numbers for this attempt, which didn't pass the threshold. One obvious issue is the low memory throughput.

ID	Estimated Speedup [%]	Function Name	Demangled Name	Duration [ms]	Runtime Improvement [ms]	Compute Throughput [%]	Memory Throughput [%]	# Registers [register/thread]	Grid Size	Block Size [block]
0	0.00	implicitGemmConv	implicitGemmConv	72.19	(0.00 ms)	65.59	67.64	31	1361250, 1, ..	32, 32, ..
1	0.00	implicitGemmConv	implicitGemmConv	114.58	(0.00 ms)	66.77	67.05	31	227813, 1, ..	32, 32, ..

Optimization 3: Rectangular Tiles for the GEMM and CPP Templates

- To address the nature of our matrices (very wide), relaxed the requirement of equal size tiles (`TILE_M=TILE_K=TILE_N`) in the original assignment 1 implementation, hoping to allow for more reuse and generally fit the problem size better.
- Wrapped the implicit GEMM kernel in a CPP template to make code work on both layers with cached constants at compile time, reducing some runtime math.
- The following shows the latency numbers for this attempt, which passed the threshold and became the base numbers for later optimization. As expected, the memory and compute throughputs increased due to the two optimization steps.

ID	Estimated Speedup [%]	Function Name	Demangled Name	Duration [ms]	Runtime Improvement [ms]	Compute Throughput [%]	Memory Throughput [%]	# Registers [register/thread]	Grid Size	Block Size [block]
0	0.00	implicitUnrollTiledG...	...implicitUnrollTiled...	25.34	(95.24 ms)	71.15	82.68	32	1361250, 1, ..	32, 12, ..
1	0.00	implicitUnrollTiledG...	...implicitUnrollTiled...	69.98	(0.00 ms)	74.56	86.14	30	303750, 1, ..	24, 24, ..

Optimization 4: New Tiled Convolution-Based Kernel for Layer 1

- Considering the matrix sizes (very small W) for Layer 1 $W(12 \times 7 \times 7 \times 1)$ $X(7 \times 7 \times 1 \times 10000 \times 66 \times 66)$, decided to try tiled convolution for layer one instead of GEMM.
- Similar to the kernel explained in class, the following **pseudo code logic** highlights the logic used for this kernel. This kernel was used for layer 1 in **all** other attempts.

```
kernel ConvTiled():
    // load filters into shared memory
    for all threads:
        load part of W into sW
    // load input tile (with halo) into shared memory
    for all channels c:
        for all tile positions (yy, xx) assigned to this thread:
            sX[c][yy][xx] = x[...]
    sync_threads()
    // initialize accumulators
    for all filters m:
        acc[m] = 0
    // Do convolution
    for all channels c:
        for all kernel positions (p, q):
            x_val = sX[c][thread_y + p][thread_x + q]
            for all filters m:
                acc[m] += sW[m][c][p][q] * x_val
    for all filters m:
        y[b][m][h_out][w_out] = acc[m]
```

- The following shows the latency numbers for this attempt, which achieved **~3.8x speedup** over attempt 3 for layer 1

ID	Estimated Speedup [%]	Function Name	Demangled Name	Duration [ms] (80.26 ms)	Runtime Improvement [ms] (0.00 ms)	Compute Throughput [%]	Memory Throughput [%]	# Registers [register/thread]	Grid Size	Block Size [block]
0	0.00	convTiled	..convTiled_float..	7.60	0.00	66.27	96.15	73	5, 9, 1000	16, 8, ..
1	0.00	implicitUnrollTiledG...	..implicitUnrollTiled...	72.66	0.00	73.50	84.63	32	303750, 1, ..	24, 24, ..

Optimization 5: New Tensor Core-Based Kernel for Layer 2

- Knowing the importance of Tensor cores, moved away from standard tiled GEMM to using WMMA api, since it is easier than lower level MMA.
- While still doing X unrolling on the fly, W is not explicitly “unrolled” anymore since its contiguous in memory in the **exact (M, C-K-K) structure** needed for the GEMM.
- This kernel can be summarized using a “pyramid-like” structure.
 - Grid of Blocks: Each block computes a $BM \times BN$ tile of the output matrix Y
 - Warps within Block, there are $WARPS_PER_BLOCK$ warps: Each warp computes a $WM \times WN$ chunk of the bigger $BM \times BN$ tile
 - WMMA Fragments within Warp: Each warp uses WMMA to compute the $WM \times WN$ chunk in smaller $WMMA_M \times WMMA_N$ fragments

- The following shows the latency numbers for this attempt, which achieved **~2.3x speedup** over attempt 4 for layer 2. However, the low **memory throughput** was obvious in the **Nsight** report.

ID	Estimated Speedup [%]	Function Name	Demangled Name	Duration [ms] (40.46 ms)	Runtime Improvement [ms] (0.00 ms)	Compute Throughput [%]	Memory Throughput [%]	# Registers [register/thread]	Grid Size
0	0.00	convTiled	..convTiled_float..	7.60	0.00	66.27	96.15	73	5
1	0.00	implicitUnrollWmm...	..implicitUnrollWmm...	32.87	0.00	66.60	48.33	63	113907

- Another observation is how low the **occupancy** was. It was limited by the amount of shared memory used, which was addressed in the next optimization, 6.

► Occupancy

Occupancy is the ratio of the number of active warps per multiprocessor to the maximum number of possible active warps. Another way to view occupancy is the percentage of the hardware's ability to process warps that is actively in use. Higher occupancy does not always result in higher performance, however, low occupancy always reduces the ability to hide latencies, resulting in overall performance degradation. Large discrepancies between the theoretical and the achieved occupancy during execution typically indicates highly imbalanced workloads.

Theoretical Occupancy [%]	43.75	Block Limit Registers [block]	8
Theoretical Active Warps per SM [warp]	28	Block Limit Shared Mem [block]	7
Achieved Occupancy [%]	43.61	Block Limit Warps [block]	16
Achieved Active Warps Per SM [warp]	27.91	Block Limit SM [block]	32

⚠ Occupancy Limiters This kernel's theoretical occupancy (43.8%) is limited by the required amount of shared memory. See the [CUDA Best Practices Guide](#) for more details on optimizing occupancy.

- Another observation is the **16% Uncoalesced Shared Accesses** when loading shared memory into the fragments at **wmma::load_matrix_sync()**

⚠ Uncoalesced Shared Accesses This kernel has uncoalesced shared accesses resulting in a total of 426467808 excessive wavefronts (41% of the total 1046121864 wavefronts). Check the L1 Wavefronts Shared Excessive table for the primary source locations. The [CUDA Best Practices Guide](#) has an example on optimizing shared memory accesses.

L1 Wavefronts Shared Excessive

Location	Value	Value (%)
mma.hpp:91 (0x153b7b859530 in implicitUnrollWmmaTC) ▶	67,432,944	16
mma.hpp:91 (0x153b7b859500 in implicitUnrollWmmaTC) ▶	67,432,944	16
mma.hpp:99 (0x153b7b859549 in implicitUnrollWmmaTC) ▶	33,716,472	8
mma.hpp:99 (0x153b7b859500 in implicitUnrollWmmaTC) ▶	33,716,472	8
mma.hpp:99 (0x153b7b8595b0 in implicitUnrollWmmaTC) ▶	33,716,472	8

Source: cuda_fp16.hpp Navigate By: L2 Theoretical Sectors Global Excessive Redo Resolve

```
# Source
86
87 //
88 // Load functions for frags of shape m16n16k16
89 //
90 __CUDA_MMA_DEVICE_DECL__ void load_matrix_sync(fragment<matrix_a,
91     __hmma_m16n16k16_ld_a((int*)&a, (const int*)p, ldm, 0);
92 }
93
94 __CUDA_MMA_DEVICE_DECL__ void load_matrix_sync(fragment<matrix_a,
95     __hmma_m16n16k16_ld_a((int*)&a, (const int*)p, ldm, 1);
96 }
97
98 __CUDA_MMA_DEVICE_DECL__ void load_matrix_sync(fragment<matrix_b, 1
```

Optimization 6: Addressing Issues with Optimization 5 Tensor Core-based Kernel

- To address the relatively **low memory throughput** from the last kernel, vectorized W loads into SMEM since they are contiguous and easy to vectorize. Instead of one thread loading 1 **float** value from global memory, load **4 floats** to exploit locality.
- To improve **occupancy** and reduce shared memory needed size per block, pack SMEM for W,X,Y into a union of structs since we only either need W,X or Y at a time. This is highlighted in the next *pseudo code logic*.

```
kernel implicitUnrollWMMA():
..
// shared memory for W,X,Y all occupy the same space
shared union {
    struct { half Wtile; half Xtile; } WX;
    struct { float Ytile;          } Y;
} smem;
// Load W and X tiles into shared memory, no need for Y
load_W_into(smem.WX.Wtile)
load_X_into(smem.WX.Xtile)
synchronize()
// Compute using WMMA, still no need for Y
wmma_multiply( using smem.WX.Wtile , smem.WX.Xtile )
synchronize()
// Store results into Y tile, no need for W and X
store_fragments_into(smem.Y.Ytile)
synchronize()
// Write Y tile to global memory, still only need Y
write_out(smem.Y.Ytile)
```

- To fix the uncoalesced shared accesses at **wmma::load_matrix_sync()**, W and X were loaded into SMEM in column major, and therefore loaded into **wmma** fragments from shared mem in column major, eliminating it. This was found through empirical testing.
- Exploited the launch config of the second kernel (the fact that **WARRPS_PER_BLOCK * WARP_SIZE** is a multiple of BN to simplify the calculations of unrolling logic indices inside the X loads into SMEM loop). This is shown below.

```
for all threads t:
    for i in elements_per_thread_X:
        idx = t + i * (WARRPS_PER_BLOCK * WARP_SIZE)
        r = idx / BN
        c = t % BN          // simplified because idx % BN == t % BN
        load_X_element_at(r, c) into shared_memory
```

- The following shows the latency numbers for the last attempt, which achieved the **best latency numbers so far**, beating the PyTorch numbers. Although the final kernel was improved, there are still more optimizations to achieve **SoL** numbers.

ID	Estimated Speedup [%]	Function Name	Demangled Name	Duration [ms] (30.87 ms)	Runtime Improvement [ms] (0.00 ms)	Compute Throughput [%]	Memory Throughput [%]	# Registers (register/thread)	Grid Size
0	0.00	convTiled	..convTiled_float..	7.60	0.00	66.26	96.14	73	5
1	0.00	implicitUnrollWmm...	..implicitUnrollWm...	23.27	0.00	58.71	75.99	64	113907

► Occupancy

Occupancy is the ratio of the number of active warps per multiprocessor to the maximum number of possible active warps. Another way to view occupancy is the percentage of the hardware's ability to process warps that is actively in use. Higher occupancy does not always result in higher performance, however, low occupancy always reduces the ability to hide latencies, resulting in overall performance degradation. Large discrepancies between the theoretical and the achieved occupancy during execution typically indicates highly imbalanced workloads.

Theoretical Occupancy [%]	50	Block Limit Registers [block]	8
Theoretical Active Warps per SM [warp]	32	Block Limit Shared Mem [block]	10
Achieved Occupancy [%]	49.82	Block Limit Warps [block]	16
Achieved Active Warps Per SM [warp]	31.88	Block Limit SM [block]	32

Note on performance numbers:

Through testing we noticed inconsistencies with values running on different GPUs. This was reported in this Piazza thread: <https://piazza.com/class/meljyfp6es4s9/post/119>